
Software Side Channels

Secure Software and Hardware Systems

Organizational Notes

Assignment 1 Grades are now released

Guest talk by Prof. Jiska Classen on Friday:

- “Novel Threats in Wireless Systems”
- Friday Dec 12th 2025, 16:00-17:00, Y3-G38

Goal of this lecture

- Familiarize you with microarchitectural components
- Provide you with understanding on how cache attacks work

Disclaimer

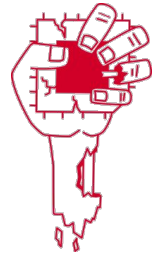
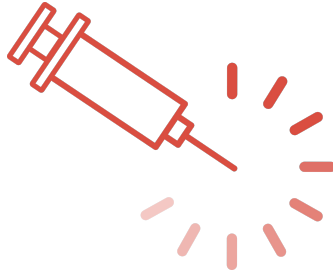
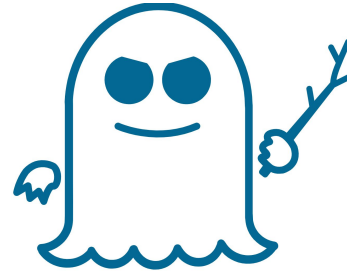
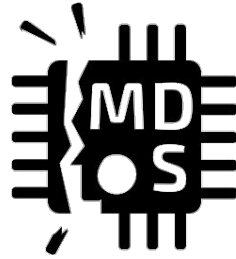
In the last lecture (hardware side channels) we looked into embedded systems.

Now we are moving back to desktop/mobile/server systems.

Overview

- Introduction
- Microarchitecture
 - Branch Prediction
 - Caches
- Cache Side-Channel Attacks
 - Flush & Reload
 - Other Attacks
- Platypus Attack
- Recap

Motivation



Microarchitectural Attacks

- Rapidly growing field since 2018
 - Initial release of Spectre/Meltdown: January 2018
 - Lot's of attacks with scary names and flashy logos
- But already relevant before, e.g.:
 - Bernstein. Cache-timing attacks on AES (2005)
 - Yarom & Falkner. "FLUSH+RELOAD: A High Resolution, Low Noise, L3 Cache Side-Channel Attack" (2014)
- Main idea:
 - Abuse hardware side-channels from software by exploiting microarchitectural components

Overview

- Introduction
- Microarchitecture
 - Branch Prediction
 - Caches
- Cache Side-Channel Attacks
 - Flush & Reload
 - Other Attacks
- Platypus Attack
- Recap

Microarchitecture

- (Assembly) Instructions lowest abstraction in programming
 - Defined by the Instruction Set Architecture
- But hardware needs to execute these instructions, including:
 - Decoding instructions
 - Interacting with memory
 - Detecting faults
- All of this is implemented in the *microarchitecture*
- Analogy: CPUs are the *interpreter* for assembly instructions. The microarchitecture is the “code” for the interpreter

Micro-Architectural Components

- Modern CPUs are complex
 - Multi-stage instruction pipelines
 - Out-of-order execution
 - Branch prediction
 - MicroOps
 - ...
- Huge gains for performance
 - But not designed for security
- In this lecture, we will look into
 - With focus on Intel CPUs

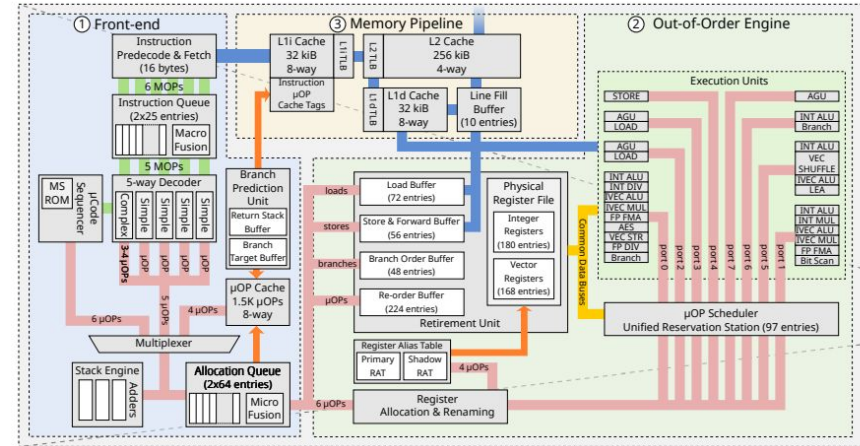
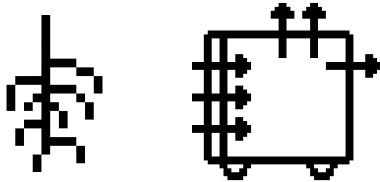


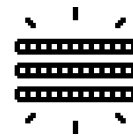
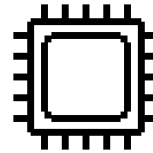
Fig. 9: a full overview of the Intel Skylake microarchitecture.

- In this lecture, we will look into 2 relevant components
 - With focus on Intel CPUs

Micro-Architectural Components

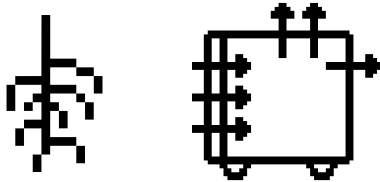


Branch Predictors

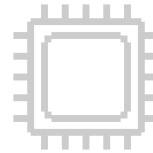


CPU Caches

Micro-Architectural Components



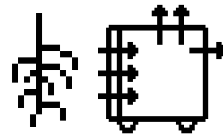
Branch Predictors



CPU Caches

Branch Predictors

- Control flow of programs is decided by branches
 - The branch prediction unit (BPU) tries to predict the target for each branch
- General good heuristic:
 - “For already encountered branches, assume the same branch target again”
- Often true in practice, e.g.:
 - Loops
 - Frequently called functions
 - Indirect branches for class methods in OOP



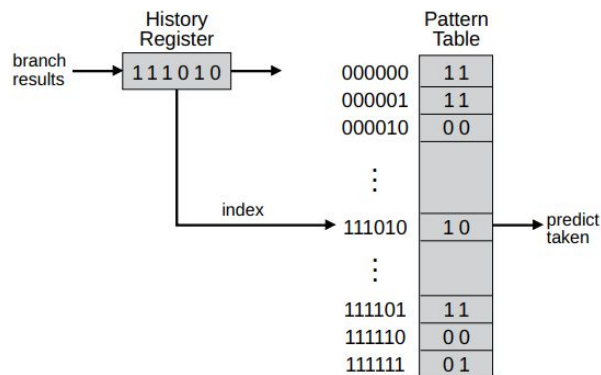
Branch Predictors

Speculative Execution

- CPU assumes that prediction of BPU as correct
 - And begins to *speculatively* execute the instructions on branch target side
 - No need to wait on computation result for data-dependent branches
 - This happens in the microarchitecture(!)
- If prediction was correct:
 - Huge performance gain
 - Results from speculative execution are architecturally committed
- If prediction was incorrect (“misprediction”):
 - Discard results of speculative execution
 - Restart pipeline at correct branch target
 - More or less same performance as if branch prediction was not in-place

Direct Branch Prediction

- Branches to fixed targets
 - May be conditional
 - In (x86) assembly, e.g.: `jmp`, `jnz`, `jz`, etc
- Performed via the “Pattern History Table” (PHT)
 - Keeps track of previously taken/non-taken history
 - Uses the resulting history as index into the pattern table
 - Optional: Also as index into branch target address cache
- Simplest form: Pattern table is a saturating 2-bit counter:
 - MSB used for prediction
 - For taken branches, increment entry
 - For not-taken, decrement entry



Indirect Branch Prediction

- Branches with targets computed at run-time
 - In (x86) assembly, e.g.: `jmp [rax], call [rdi]`
 - More complicated to predict
- Prediction via:
 - Branch History Buffer (BHB)
 - Branch Target Buffer (BTB)
- Creates tags for prediction by combining:
 - Source
 - Destination
 - History

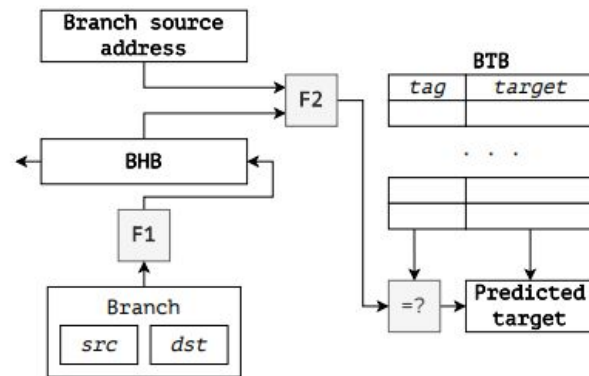
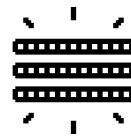
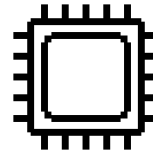


Figure 1: A simplified diagram of indirect branch prediction logic.

Micro-Architectural Components

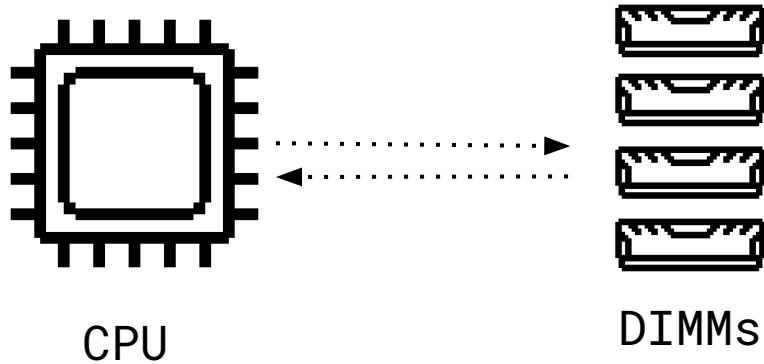


Branch Predictors



CPU Caches

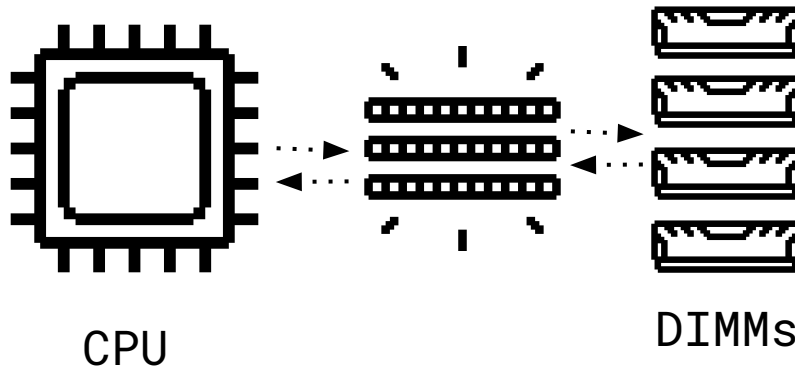
The Problem



Accessing data from memory is a bottleneck

- Data transfer via bus
- Memory controller to map physical addresses to DIMMs
- Complex organization of memory on DIMM (Bank, Ranks, Rows)

Solution: CPU Caches



CPU Caches:

- Microarchitectural component between memory and CPU
- Caches memory contents
- Usually implemented with SRAM
 - Small sizes
 - Fast access types

CPU Caches: Main ideas

- Previously accessed data & instruction:
 - Have a high likelihood to be accessed again
 - Let's keep them close to the CPU!
- When data is loaded from DIMM, bring it into cache
 - Subsequent operations are on value in cache
 - Huge performance optimization
- When cache is full and new values are loaded from memory:
 - Write data back from cache to memory ("evict" old values)

Cache Organization

Types of Caches:

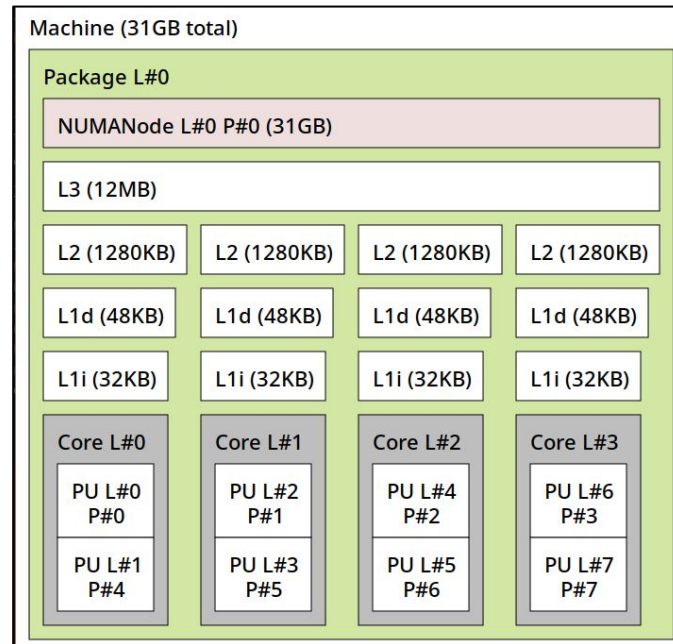
- I-Cache: Instruction Cache, for executed code
- D-Cache: Data Cache, for accessed data

Cache Granularity:

- Organized in “Lines”
- Usual cache line size: 64kb
- Single memory access brings (or evicts) full line from cache

Cache Hierarchy

- L1: Very Small & Very Fast
 - Typically located directly on the CPU
- L2: Small & Fast
 - Typically located on CPU or in proximity
- L3: Large & Slow
 - Typically shared across multiple cross
- Memory: Very large & very slow



Example 1stopo output



Observation

CPU caches introduce a timing side channel

Data in cache:
Fast Access (Cache “Hit”)



Data not in cache:
Slow Access (Cache “Miss”)



Overview

- Introduction
- Microarchitecture
 - Branch Prediction
 - Caches
- Cache Side-Channel Attacks
 - Flush & Reload
 - Other Attacks
- Platypus Attack
- Recap

Cache Attacks: Flush+Reload

- Originally leaks from a victim to a spy process
 - Assumes shared memory between victim and spy
 - Can even work via L3
 - Shared code and libraries(!)
- Strategy:
 - 1) Spy flushes cache
 - 2) Victim carries out operations
 - 3) Spy probes cache and measures time:
 - Fast access: victim brought value into cache

```
int probe(char *adrs) {  
    volatile unsigned long time;  
    asm __volatile__ (  
        " mfence                \n"  
        " lfence                \n"  
        " rdtsc  \n"  
        " lfence                \n"  
        " movl %%eax, %%esi      \n"  
        " movl (%1), %%eax *ADRS \n"  
        " lfence                \n"  
        " rdtsc  \n"  
        " subl %%esi, %%eax      \n"  
        " clflush 0(%1)         \n"  
        : "=a" (time)  
        : "c" (adrs)  
        : "%esi", "%edx");  
    return time < threshold;  
     OR  ?  
}
```

Flush & Reload: Live Demo

Based on: <https://github.com/casperIITB/Flush-Reload-attack.git>

Other Cache Attacks

- Flush & Flush
 - Mechanism: Flush takes longer when data is cached
 - Use-case: Same as flush and reload, but more stealthy
- Evict & Reload
 - Mechanism: Use cache eviction policies to evict sensitive data from cache
 - Use-case: `clflush` (or similar) instruction unavailable, shared memory available
- Prime & Probe
 - Mechanism:
 - (1) Attacker primes cache by filling all values.
 - (2) If later access is slow, victim accessed relevant cache line in-between
 - Use-case: `clflush` (or similar) instruction unavailable, shared memory unavailable

But what can these attacks do?

- Breaking cryptography by learning:
 - Which parts of code was executed (e.g., Square-and-Multiply in RSA)
 - Which parts of data was accessed (e.g., look-up tables in AES)
- Building covert channels
 - Transmitting information between two processes not allowed to communicate
- Exfiltrating data during transient execution attacks
 - We can probe whether certain data ended up in cache

Constant Time Operation

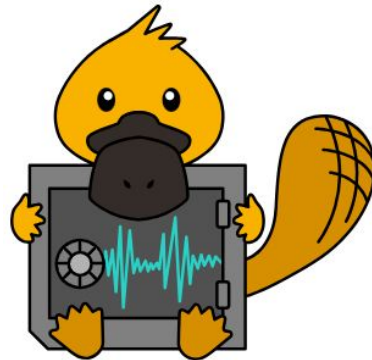
- Ensure that algorithm run time is not dependent on data/inputs
 - Main defense against timing attacks
- Requirements:
 - No data-dependent branches
 - No data-dependent memory accesses ← *MORE LATER!*

Overview

- Introduction
- Microarchitecture
 - Branch Prediction
 - Caches
- Cache Side-Channel Attacks
 - Flush & Reload
 - Other Attacks
- Platypus Attack
- Recap

Beyond the Cache

- As with hardware, cache timing-based side channels are just one possibility
- Recent, different example: Platypus Attack
<https://platypusattack.com/>
- Uses Intel's Running Average Power Limit (RAPL) functions to measure power consumption of target operations
 - Similar to traditional power side channels



RAPL?

RAPL on Linux

There was at one point *four* ways to read RAPL results on Linux:

- Reading the files under `/sys/class/powercap/intel-rapl/intel-rapl:0` using the powercap interface. This required no special permissions, and was introduced in Linux 3.13. Despite my pointing out lack of permissions was a problem at the time of introduction, nothing was done about this until Linux 5.10 where the security issue was noticed (949dd0104c496fa7c14991a23c03c62e44637e71) and limitations were added.
- Using the `perf_event` interface with Linux 3.14 or newer. This requires root or a `paranoid` less than 1 (as do all system wide measurements with `-a`) `sudo perf stat -a -e "power/energy-cores/" /bin/ls` Available events can be found via `perf list` or under `/sys/bus/event_source/devices/power/events/`
- Using raw-access to the underlying MSR under `/dev/msr`. This requires root. Some HPC tools will attempt to use this interface to bypass the kernel, but the kernel developers want to make raw MSR access go away as there are a lot of questionable things you can do to a system with raw MSR access. There have been attempts to make alternate drivers that whitelist the MSRs that can be used but they are probably going away in the future too.
- An AMD RAPL driver without premissions checks was available under the kernel `hwmon` interface, but it was removed in Linux 5.13 (9049572fb) also due to security concerns.

Note that you cannot get readings for individual processes, the results are for the entire CPU socket.

RAPL?

ENTRYPOINT FOR PLATYPUS



RAPL on Linux

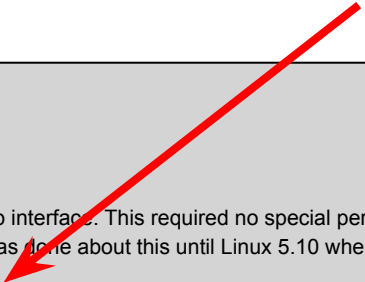
There was at one point *four* ways to read RAPL results on Linux:

- Reading the files under `/sys/class/powercap/intel-rapl/intel-rapl:0` using the powercap interface. This required no special permissions, and was introduced in Linux 3.13. Despite my pointing out lack of permissions was a problem at the time of introduction, nothing was done about this until Linux 5.10 where the security issue was noticed (949dd0104c496fa7c14991a23c03c62e44637e71) and limitations were added.
- Using the `perf_event` interface with Linux 3.14 or newer. This requires root or a paranoid less than 1 (as do all system wide measurements with `-a`) `sudo perf stat -a -e "power/energy-cores/" /bin/ls` Available events can be found via `perf list` or under `/sys/bus/event_source/devices/power/events/`
- Using raw-access to the underlying MSR's under `/dev/msr`. This requires root. Some HPC tools will attempt to use this interface to bypass the kernel, but the kernel developers want to make raw MSR access go away as there are a lot of questionable things you can do to a system with raw MSR access. There have been attempts to make alternate drivers that whitelist the MSR's that can be used but they are probably going away in the future too.
- An AMD RAPL driver without premissions checks was available under the kernel `hwmon` interface, but it was removed in Linux 5.13 (9049572fb) also due to security concerns.

Note that you cannot get readings for individual processes, the results are for the entire CPU socket.

RAPL?

STILL RELEVANT FOR TRUSTED
EXECUTION/SECURE ENCLAVES



RAPL on Linux

There was at one point *four* ways to read RAPL results on Linux:

- Reading the files under `/sys/class/powercap/intel-rapl/intel-rapl:0` using the powercap interface. This required no special permissions, and was introduced in Linux 3.13. Despite my pointing out lack of permissions was a problem at the time of introduction, nothing was done about this until Linux 5.10 where the security issue was noticed (949dd0104c496fa7c14991a23c03c62e44637e71) and limitations were added.
- Using the `perf_event` interface with Linux 3.14 or newer. **This requires root or a paranoid less than 1** (as do all system wide measurements with `-a`) `sudo perf stat -a -e "power/energy-cores/" /bin/ls` Available events can be found via `perf list` or under `/sys/bus/event_source/devices/power/events/`
- Using raw-access to the underlying MSRs under `/dev/msr`. **This requires root.** Some HPC tools will attempt to use this interface to bypass the kernel, but the kernel developers want to make raw MSR access go away as there are a lot of questionable things you can do to a system with raw MSR access. There have been attempts to make alternate drivers that whitelist the MSRs that can be used but they are probably going away in the future too.
- An AMD RAPL driver without premissions checks was available under the kernel `hwmon` interface, but it was removed in Linux 5.13 (9049572fb) also due to security concerns.

Note that you cannot get readings for individual processes, the results are for the entire CPU socket.

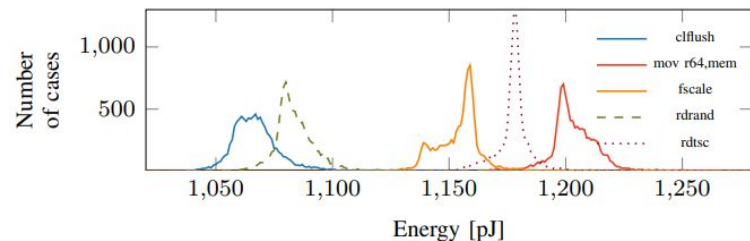
Measuring Instruction Energy Consumption via RAPL

- RAPL allows to measure the consumed energy over sampling period
 - Update intervals up to 50 μ s
- Energy is closely related to power consumption:
 - Cumulative consumed power over time
- RAPL measurements can target four different domains
 - package (PKG), power planes (PP0 and PP1), and DRAM
- Similar interfaces available for AMD

Measuring Instruction Energy Consumption via RAPL

Instruction	Xeon Silver 4214	i7-6700K	i7-8650U
nop	0.1795 nJ	0.1189 nJ	0.0843 nJ
inc r64	0.1795 nJ	0.1208 nJ	0.0858 nJ
xor r64, r64	0.1795 nJ	0.1209 nJ	0.0849 nJ
mov r64, mem	0.1868 nJ	0.1247 nJ	0.0840 nJ
imul r64, r64	0.1798 nJ	0.1169 nJ	0.0887 nJ
fscale	0.1867 nJ	0.1182 nJ	0.0877 nJ
rdrand r64	0.1797 nJ	0.1129 nJ	0.0982 nJ
rdtsc	0.1864 nJ	0.1189 nJ	0.0848 nJ
clflush mem	0.1865 nJ	0.1129 nJ	0.1018 nJ
aesenc xmm, xmm	0.1794 nJ	0.1188 nJ	0.0946 nJ

Average Observed Energy Consumption.



Histogram for different instructions (i7-6700K).

Platypus: Example Attacks

Unprivileged Attacker:

- Breaking KASLR within 20 seconds
- Using the (unprivileged) powercap interface directly
 - And requires Intel TSX for fault suppression
 - Not possible on modern systems

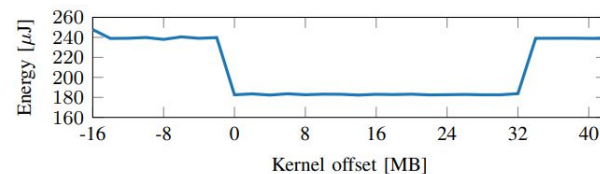


Fig. 10: Power consumption when transiently accessing kernel addresses. If a kernel page is not mapped, the access triggers an entire page-table walk, consuming more power.

Privileged Attacker:

- Side-channel attack on execution in SGX enclave
- Recovering RSA private keys within 100 minutes
- Leaking of keys for AES-NI between 26-277 hours

Overview

- Introduction
- Microarchitecture
 - Branch Prediction
 - Caches
- Cache Side-Channel Attacks
 - Flush & Reload
 - Other Attacks
- Platypus Attack
- Recap

Recap

Looked into two key microarchitectural components

- “What” are the root causes causing microarchitectural attacks

Took a deep dive into cache side-channel attacks

- “How” can cache states leak information?
- “What” can we do as attackers with this side channel?

Investigated Platypus as a “hardware from software” side channel

- “How” can concepts like power analysis effect desktop systems?

Where can I learn more?

- As usual: Referenced papers in this presentation
- Proof-of-concept implementations on Github
 - Read, Execute, Understand
- Side Channel Security Youtube Channel:
 - <https://www.youtube.com/@SideChannelSecurity/videos>